

Claviger - Guide pédagogique du code et de l'architecture

Version étudiée : branche main fournie le 7 septembre 2026

Nature du document : rétro-ingénierie pédagogique du code de la V1

But : comprendre suffisamment le système pour le diagnostiquer, le maintenir et intervenir dessus sans devoir relire l'intégralité des sources à chaque fois.

Ce document n'est ni une API reference ni une liste exhaustive des fonctions. Il explique les mécanismes qui structurent réellement Claviger : démarrage, intégration Discord, composition des dépendances, cycle des interactions, UI, réconciliation des rôles, persistance SQLite, synchronisation des catalogues, politiques, autorisation, reporting et tests.

1. Comment lire ce guide

Claviger contient plusieurs milliers de lignes de code réparties entre le code applicatif et les tests. Une lecture linéaire fichier par fichier est possible, mais elle n'est pas la manière la plus efficace de construire un modèle mental du système.

La bonne question n'est pas :

« Que fait chaque fonction ? »

mais plutôt :

« Quand un utilisateur déclenche une action, quelles couches prennent successivement la responsabilité, quelles données transitent entre elles, et où se produit l'effet de bord réel ? »

Ce guide suit donc les **flux** avant les fichiers.

Les quatre notions à garder en tête pendant toute la lecture sont :

1. **Discord est l'interface**, pas le coeur métier.
2. **Un coordinator orchestre**, il ne devrait pas contenir tout le calcul.
3. **Un planner décide**, mais ne modifie rien.
4. **Un executor applique** les effets de bord déjà décidés.

La V1 a été volontairement orientée vers cette séparation afin d'éviter un bot construit autour de handlers Discord monolithiques. La rétrospective du projet documente explicitement ce choix : les commandes devaient rester des adaptateurs et la logique réutilisable devait vivre dans des services.

1.1 La carte mentale en trente secondes

```

Discord
|
| slash command / modal submit
v
commands/ et ui/
|
| transforme une Interaction Discord en appel applicatif
v
coordinator
|
|--> questionnaire service --> repository --> SQLite
|
|--> planner -----> plan immuable
|
|--> executor -----> RoleManager --> API Discord

En parallèle :
policy resolver --> règles du serveur
report service  --> logs / forum administratif
  
```

Si un bug concerne **ce que l'utilisateur voit**, commencer par `commands/` ou `ui/`.

Si un bug concerne **ce qui devrait être ajouté ou retiré**, commencer par le **planner**.

Si le plan est juste mais que Discord n'applique pas l'opération, commencer par l'**executor** puis **RoleManager**.

Si une option n'apparaît pas dans un questionnaire, regarder le **questionnaire service**, le catalogue et sa synchronisation.

Si la base est incohérente ou non prête, regarder **database/** et **repositories/**.

2. Les bibliothèques utilisées et ce qu'elles apportent

Le fichier **pyproject.toml** décrit peu de dépendances de production. C'est un bon point pédagogique : presque tout ce que fait Claviger peut être rattaché à une bibliothèque précise.

Bibliothèque	Rôle dans Claviger	Pourquoi elle existe ici
discord.py	Client Discord, slash commands, interactions, modales, rôles, salons, embeds	Fournit le modèle objet et le protocole nécessaire pour parler à Discord sans implémenter soi-même l'API Gateway/REST.
aiosqlite	Accès SQLite asynchrone	Évite de bloquer la boucle asynchrone du bot pendant les opérations SQL et fournit une API proche de sqlite3.
python-dotenv	Chargement local des variables d'environnement	Permet d'utiliser un .env en développement tout en gardant la configuration hors du code.
pytest	Tests	Fournit les fixtures, assertions et mécanismes de collecte utilisés par la suite de tests.
pytest-asyncio	Tests des fonctions async	Permet à pytest d'exécuter les coroutines employées par Discord et la DB.
Ruff	Lint/imports	Détecte rapidement erreurs simples, imports inutilisés et incohérences de style.
setuptools	Packaging	Rend le package claviger installable et expose la commande console claviger.

2.1 Pourquoi l'asynchrone est partout

Discord est un système réseau événementiel. Le bot attend en permanence des événements puis effectue des appels réseau : recevoir une interaction, envoyer un message, ajouter un rôle, lire un salon, etc.

En Python, **discord.py** repose sur **asyncio**. Une fonction telle que :

```
async def send(...):
    await channel.send(message)
```

signifie essentiellement :

« Cette opération peut attendre le réseau. Pendant cette attente, rends la main à la boucle d'événements afin qu'elle puisse traiter autre chose. »

Le mot-clé **await** ne crée pas un thread. Il suspend la coroutine courante jusqu'à ce que l'opération attendue puisse reprendre.

C'est aussi la raison du choix **aiosqlite**. Une opération SQL effectuée au milieu d'un handler Discord ne doit pas bloquer inutilement le reste du bot.

3. Du terminal jusqu'à Discord : main.py

Le point d'entrée est volontairement minuscule :

```
from claviger.bot import ClavigerBot
from claviger.config import get_discord_token
```

```
def main() -> None:
    token = get_discord_token()
    bot = ClavigerBot()
    bot.run(token)
```

Ce fichier joue le rôle de **bootstrap minimal**.

Il réalise trois opérations :

1. lire le secret nécessaire à la connexion ;
2. construire l'application ;
3. laisser **discord.py** prendre le contrôle de la boucle d'exécution.

Il ne construit pas lui-même les services et ne contient aucune logique métier. Cette sobriété est utile : lorsqu'un démarrage échoue avant même la connexion Discord, il n'y a que très peu d'endroits à inspecter.

3.1 Pourquoi **bot.run(token)** paraît synchrone alors que le projet est asynchrone

ClavigerBot hérite de **discord.Client**. La méthode **run()** appartient à **discord.py**.

Elle masque volontairement le bootstrap asynchrone : création/gestion de la boucle événementielle, connexion au Gateway Discord, reconnexion et cycle de vie du client.

Le code applicatif n'appelle donc pas manuellement **asyncio.run()**.

3.2 Pourquoi le package expose aussi une commande **claviger**

Dans **pyproject.toml** :

```
[project.scripts]
claviger = "claviger.main:main"
```

Une fois le projet installé, l'outil de packaging peut créer une commande console nommée **claviger** qui appelle **claviger.main.main**.

Cela découple le lancement de l'emplacement physique du fichier. Le programme n'a pas besoin d'être démarré avec une commande fragile du type **python src/claviger/main.py**.

4. bot.py : le composition root

bot.py est l'un des fichiers les plus importants pour comprendre Claviger. Il ne contient pourtant presque aucune règle métier.

Sa responsabilité est de **composer l'application**.

Dans un projet disposant d'un framework d'injection de dépendances, une partie de ce travail pourrait être automatisée. Claviger le fait explicitement.

Exemple réel :

```
self.interest_catalog_repository = InterestCatalogRepository(
    self.database,
)

self.member_interest_questionnaire_service = MemberInterestQuestionnaireService(
    repository=self.interest_catalog_repository,
)

self.member_role_planner_service = MemberRolePlannerService()

self.member_role_executor_service = MemberRoleExecutorService(
    role_manager=self.role_manager,
)

self.member_workflow_coordinator_service = MemberWorkflowCoordinatorService(
    questionnaire_service=self.member_interest_questionnaire_service,
    planner_service=self.member_role_planner_service,
    executor_service=self.member_role_executor_service,
)
```

L'intérêt pédagogique est important : on peut lire ce bloc comme un schéma de dépendances.



4.1 Pourquoi construire les dépendances ici plutôt que dans les commandes

Une commande pourrait techniquement faire ceci à chaque invocation :

```

repository = InterestCatalogRepository(...)
service = MemberInterestQuestionnaireService(repository)
...

```

Mais cela poserait plusieurs problèmes :

- la commande connaîtrait la construction de toute l'infrastructure ;
- les tests auraient plus de mal à substituer un mock ;
- les mêmes objets seraient reconstruits inutilement ;
- le changement d'une dépendance obligerait à modifier plusieurs points d'entrée.

À la place, **bot.py** construit les objets une fois puis **injecte** les objets déjà prêts dans les factories de commandes.

Ce n'est pas un framework DI. C'est de l'injection de dépendances manuelle et explicite.

4.2 Pourquoi ClavigerBot hérite de discord.Client

```
class ClavigerBot(discord.Client):
```

`discord.Client` est le composant qui représente la connexion Discord elle-même.

Il expose notamment :

- le cache des guilds et channels connus ;
- l'utilisateur connecté (`self.user`) ;
- les événements de cycle de vie ;
- les appels REST nécessaires à certaines opérations.

Claviger ajoute autour de ce client son graphe de services et sa `CommandTree`.

4.3 Intents : ce que le bot demande à Discord

```

intents = discord.Intents.default()
super().__init__(intents=intents)

```

Les intents indiquent quelles catégories d'événements le bot souhaite recevoir du Gateway Discord.

La V1 utilise les intents par défaut et n'active pas ici de privileged intent supplémentaire. C'est cohérent avec les fonctionnalités actuelles : les workflows sont déclenchés par des interactions et manipulent les objets accessibles via la guild.

Si une future fonctionnalité dépend d'événements membres plus riches ou du contenu des messages, le choix des intents deviendra une vraie décision de conception et de configuration côté Developer Portal Discord.

5. La CommandTree : comment les slash commands existent réellement

Dans `ClavigerBot.__init__` :

```
self.tree = app_commands.CommandTree(self)
```

`CommandTree` est la structure `discord.py` qui contient les **application commands** : slash commands et groupes associés.

Créer une fonction décorée avec `@app_commands.command` ne suffit pas à la faire apparaître partout. Clavier construit les commandes, les ajoute à l'arbre, puis synchronise cet arbre avec Discord.

5.1 Pourquoi les commandes sont enregistrées dans `bot.py`

Exemple :

```
self.tree.add_command(
    create_member_command(
        self.policy_resolver,
        self.database_status_service,
        self.member_workflow_coordinator_service,
    ),
    guild=guild,
)
```

`create_member_command(...)` est une **factory** : elle reçoit les dépendances dont la commande a besoin et retourne un objet `app_commands.Command`.

Cette forme peut sembler inhabituelle lorsqu'on vient d'un bot où toutes les commandes sont directement des méthodes décorées d'une grosse classe. Ici elle permet précisément de garder une commande testable sans lui faire rechercher ses dépendances globalement.

La commande dépend de :

- `PolicyResolver` ;
- `DatabaseStatusService` ;
- `MemberWorkflowCoordinatorService`.

Elle ne dépend pas directement de `SQLite`, du `repository` ou du `RoleManager`.

5.2 Pourquoi `discord.Object(id=...)` est utilisé

```
guild = discord.Object(id=self.guild_id)
```

Pour enregistrer ou synchroniser une commande dans un guild donné, Discord n'a pas besoin de tout l'objet `Guild`. Il lui faut surtout son identifiant.

`discord.Object` est un objet léger contenant un snowflake Discord. Il permet ici de préciser la portée sans devoir effectuer une requête réseau juste pour construire la commande.

5.3 Pourquoi les commandes sont synchronisées dans `setup_hook`

```
async def setup_hook(self) -> None:
    guild = discord.Object(id=self.guild_id)
    synced = await self.tree.sync(guild=guild)
```

`setup_hook()` est appelé pendant la phase de préparation du client, avant l'état pleinement `ready`.

Clavier l'utilise pour synchroniser la `CommandTree` dans le guild configuré.

Le choix **guild-scoped** est pratique pendant le développement et pour un bot destiné d'abord à un serveur : les commandes de guild sont généralement disponibles plus rapidement que des commandes globales et leur périmètre est explicite.

5.4 `on_ready()` n'est pas le constructeur

```
async def on_ready(self) -> None:
    ...
```

`on_ready` est un callback événementiel : Discord indique que le client est prêt.

Il ne faut pas raisonner comme si cette méthode ne pouvait être appelée qu'une seule fois. Une reconnexion peut provoquer de nouveaux événements de disponibilité. C'est une des raisons pour lesquelles les mutations métier au démarrage ont été évitées : les callbacks réseau ne sont pas un bon endroit pour cacher des modifications persistantes irréversibles.

6. Comprendre une interaction Discord

Une slash command reçoit un objet central :

```
interaction: discord.Interaction
```

Une **Interaction** regroupe le contexte de l'action :

- l'utilisateur (`interaction.user`) ;
- la guild (`interaction.guild`) ;
- le channel (`interaction.channel`) ;
- l'API de réponse (`interaction.response`) ;
- les followups ;
- des identifiants permettant à Discord de rattacher la réponse à l'action initiale.

Claviger exploite cet objet **dans les boundaries Discord**, puis tente de ne pas le propager jusque dans les planners.

C'est une frontière essentielle.

6.1 `interaction.response.send_message()`

Pour une réponse immédiate :

```
await interaction.response.send_message(
    "Cette commande doit être utilisée sur un serveur.",
    ephemeral=True,
)
```

`ephemeral=True` signifie que la réponse n'est visible que par l'utilisateur ayant déclenché l'interaction.

C'est adapté aux erreurs de contexte, résultats administratifs et questionnaires : le salon public n'est pas pollué par des détails de validation.

6.2 Une interaction ne peut pas rester sans réponse indéfiniment

Discord impose une fenêtre courte pour accuser réception d'une interaction. Lorsqu'un traitement peut prendre plus de temps, le code utilise :

```
await interaction.response.defer(
    ephemeral=True,
    thinking=True,
)
```

`defer()` signifie :

« J'ai reçu l'interaction, le résultat arrive. »

Après ce `defer`, le code ne répond plus avec un second `interaction.response.send_message()` comme si rien n'avait été envoyé. Il modifie la réponse différée ou utilise un `followup`.

Deux formes sont visibles dans Claviger :

```
await interaction.edit_original_response(content="...")
```

et :

```
await interaction.followup.send("...", ephemeral=True)
```

6.3 Pourquoi les modales utilisent souvent `edit_original_response`

La soumission d'une modal n'est pas un message Discord classique que l'on peut éditer comme origine.

Le code Noctis documente explicitement le mécanisme :

```
await interaction.response.defer(
    ephemeral=True,
    thinking=True,
)
```

```
await interaction.edit_original_response(
    content="Mise à jour ...",
)
```

Le `thinking=True` crée une réponse différée éditable. Cela permet d'afficher un état « en cours », puis de remplacer ce contenu par le succès ou l'échec.

7. Une slash command doit rester une boundary fine

Prenons `/membre`.

Le handler réalise essentiellement les étapes suivantes :

```
1. Vérifier que l'interaction vient d'une guild
2. Vérifier que la DB est prête
3. Résoudre la policy
4. Vérifier le salon autorisé
5. Demander au coordinator de construire le questionnaire
6. Construire la modal
7. Envoyer la modal à Discord
```

Ce qui est remarquable ici est surtout ce qu'il **ne fait pas** :

- il ne lit pas directement SQLite ;
- il ne calcule pas les différences de rôles ;
- il ne boucle pas sur les rôles à ajouter ;
- il ne connaît pas la structure SQL du catalogue.

Cela permet de lire le handler comme un **contrat d'entrée** plutôt que comme tout le programme.

7.1 Pourquoi valider `interaction.guild is None`

Une application command peut théoriquement être invoquée dans des contextes où il n'y a pas de guild exploitable.

Le reste du workflow dépend de concepts propres à un serveur : rôles, salons, policy par guild. Le handler coupe donc immédiatement le flux si cette précondition n'est pas satisfaite.

C'est un exemple de **fail fast** : refuser tôt plutôt que laisser une `AttributeError` apparaître plusieurs couches plus bas.

7.2 Pourquoi vérifier l'état de la DB avant le questionnaire

```
database_status = await database_status_service.check()

if database_status.state != DatabaseState.READY:
    ...
    return
```

Le questionnaire dépend du catalogue stocké en base. Il vaut mieux répondre explicitement « indisponible » que poursuivre jusqu'à une erreur SQL plus loin.

Le service de statut ne modifie rien. Il diagnostique seulement : absente, non initialisée, à migrer, trop récente, indisponible ou prête.

7.3 Pourquoi le salon fait partie de la règle fonctionnelle

```
channel_name = getattr(interaction.channel, "name", None)

if channel_name != policy.salutations_channel_name:
    ...
```

Ce n'est pas une simple préférence d'interface. Dans la V1, le parcours Discord fait partie du workflow : la commande doit être utilisée dans le salon prévu après consultation des informations.

Le nom du salon est donc lu dans une `GuildPolicy` et non caché comme constante dans la commande.

8. Les modales : traduire le domaine dans le format attendu par Discord

`MemberQuestionnaireModal` et `NoctisQuestionnaireModal` héritent de `CatalogSelectionModal`.

Ce découpage répond à un problème concret : les deux workflows ont besoin d'afficher une liste d'options de catalogue, de restaurer les choix existants et de contrôler que seul l'utilisateur ayant ouvert le formulaire peut le soumettre.

Au lieu de recopier toute la mécanique Discord, la base générique reçoit des objets simples :

```
@dataclass(frozen=True, slots=True)
class CatalogSelectionOption:
    key: str
    label: str
    description: str | None = None
    emoji: str | None = None
    selected: bool = False
```

Le domaine fournit donc une **description de l'option**. La base UI la transforme ensuite en objet Discord.

8.1 CheckboxGroupOption : ce que Discord attend

Extrait réel :

```
checkbox_options = [
    discord.CheckboxGroupOption(
        label=_build_option_label(option),
        value=option.key,
        description=_build_option_description(option),
        default=option.selected,
    )
    for option in options
]
```

Discord ne connaît pas **RoleChannelCatalogEntry**. Il attend des composants UI avec un label, une valeur, éventuellement une description et un état par défaut.

Claviger effectue donc une conversion :

```
Model métier/catalogue
|
v
CatalogSelectionOption
|
v
Discord CheckboxGroupOption
```

La **value** envoyée à Discord est la clé stable du catalogue, pas le texte affiché.

C'est important : changer un label « Musique » en « Musique & concerts » ne doit pas changer l'identité logique de l'option.

8.2 Pourquoi le code tronque les chaînes

Les constantes :

```
MAX_OPTION_LABEL_LENGTH = 100
MAX_OPTION_VALUE_LENGTH = 100
MAX_OPTION_DESCRIPTION_LENGTH = 100
```

ne sont pas des règles métier de Claviger. Elles protègent l'adaptateur contre les contraintes de composants Discord.

C'est exactement le type de logique qui a sa place dans **ui/** : elle existe parce que l'interface externe impose un format.

8.3 Pourquoi custom_id existe

```
self.selection_group = discord.ui.CheckboxGroup(
    custom_id=checkbox_custom_id,
    ...
)
```

Discord doit pouvoir identifier le composant interactif lorsque l'utilisateur soumet le formulaire.

Le **custom_id** n'est pas un ID de rôle ni une clé de base de données. C'est l'identité du composant d'interface dans le protocole d'interaction Discord.

8.4 Pourquoi vérifier actor_id

```
async def interaction_check(self, interaction):
    if interaction.user.id == self.actor_id:
        return True
    ...
    return False
```


La modal appartient à l'utilisateur qui l'a ouverte.

Même si Discord limite déjà beaucoup de scénarios d'interaction, Clavier rend explicitement la propriété du formulaire. La sécurité devient lisible dans le code : une soumission par un autre acteur est refusée à la boundary UI.

9. /membre : le flux complet, de l'ouverture à la mutation

Le meilleur moyen de comprendre l'architecture est de suivre un cas réel.

9.1 Ouverture du questionnaire

```

/membre
|
v
create_member_command.member()
|
+--> DatabaseStatusService.check()
+--> PolicyResolver.resolve(guild_id)
+--> contrôle du salon
|
v
MemberWorkflowCoordinatorService.build_questionnaire()
|
+--> vérifie la policy
+--> résout le rôle Membre
|
v
MemberInterestQuestionnaireService.build_for_member()
|
v
InterestCatalogRepository.list_for_guild()
|
v
MemberInterestQuestionnaire
|
v
MemberQuestionnaireModal
|
v
interaction.response.send_modal()

```

À ce moment, **aucun rôle n'a été modifié**.

Le bot ne fait qu'observer : policy, catalogue et rôles actuels du membre.

9.2 Le questionnaire restaure l'état réel du membre

Dans `MemberInterestQuestionnaireService` :

```

member_role_ids = {role.id for role in member.roles}

selected_interest_keys = tuple(
    interest.catalog_key
    for interest in interests
    if interest.role_id in member_role_ids
)

```

Le catalogue indique quels rôles correspondent aux choix. Les rôles effectivement présents sur le membre indiquent quelles cases doivent être précochées.

La DB ne stocke donc pas une seconde copie de la sélection utilisateur. Pour ce workflow, Discord reste la source de vérité de l'état réellement attribué.

9.3 Pourquoi seules certaines entrées du catalogue sont exposées

Le service filtre les entrées avant de les présenter :

```

return (
    interest.enabled
    and interest.is_configured
    and interest.discord_present
    and interest.role_manageable
    and interest.channel_present
    and interest.mapping_valid
    and interest.matches_policy
)

```

Une ligne existant en SQLite ne signifie pas automatiquement « option disponible ».

Cette distinction est fondamentale : la base conserve aussi l'historique et l'état de synchronisation. L'UI ne doit exposer que ce qui est actuellement utilisable.

10. Soumettre /membre : pourquoi l'état est reconstruit

Une modal peut rester ouverte plusieurs minutes.

Pendant ce temps :

- un rôle peut être supprimé ;
- une entrée peut être désactivée ;
- le catalogue peut être resynchronisé ;
- le membre peut recevoir ou perdre un rôle par un autre moyen.

Claviger ne fait donc pas confiance aveuglément à l'objet questionnaire créé à l'ouverture.

Dans le coordinator :

```
questionnaire = await self.questionnaire_service.build_for_member(
    guild_id=guild.id,
    member=member,
)

current_role_ids = {role.id for role in member.roles}

plan = self.planner_service.build_plan(
    questionnaire=questionnaire,
    selected_interest_keys=selected_interest_keys,
    member_role_ids=current_role_ids,
    member_role_id=member_role.id,
)
```

Cette reconstruction est un garde-fou contre l'état UI périmé.

Le formulaire dit seulement :

« Voici les clés que l'utilisateur vient de sélectionner. »

Le serveur redécide ensuite, à partir de l'état courant, si ces clés sont toujours légitimes et quelles mutations sont réellement nécessaires.

11. Le pattern Planner / Executor : décider avant d'agir

C'est probablement le choix de conception le plus important de la V1.

Le `MemberRolePlannerService` ne dépend pas de `discord.py`. Il reçoit des données et retourne un plan :

```
@dataclass(frozen=True, slots=True)
class MemberRolePlan:
    add_role_ids: tuple[int, ...] = ()
    remove_role_ids: tuple[int, ...] = ()
```

Cette classe ne sait pas **comment** ajouter un rôle. Elle décrit seulement **ce qui devrait arriver**.

11.1 Pourquoi un plan plutôt qu'une mutation directe

Une implémentation plus courte aurait pu ressembler conceptuellement à :

```
pour chaque case cochée:
    ajouter le rôle
pour chaque case décochée:
    retirer le rôle
```

Le problème est que calcul et effet de bord seraient mélangés.

Avec un plan :

```
Etat actuel + état souhaité
    |
    v
  PLANNER
    |
    v
Plan de mutations
```



On obtient plusieurs bénéfices :

- le calcul est testable sans réseau ;
- on peut inspecter le plan avant de l'appliquer ;
- l'executor peut valider l'ensemble des cibles avant la première mutation ;
- la logique de différence n'est pas contaminée par les exceptions de l'API Discord.

11.2 La réconciliation, concrètement

Le planner transforme les clés sélectionnées en IDs de rôles désirés :

```

desired_interest_role_ids = tuple(
    interest.role_id
    for interest in questionnaire.interests
    if interest.catalog_key in selected_keys
)

```

Puis il calcule les deltas.

Pour ajouter :

```

for role_id in desired_interest_role_ids:
    if role_id not in member_role_ids:
        add_role_ids.append(role_id)

```

Pour retirer :

```

for interest in questionnaire.interests:
    if (
        interest.role_id in member_role_ids
        and interest.role_id not in desired_interest_role_id_set
    ):
        remove_role_ids.append(interest.role_id)

```

La nuance capitale est la boucle sur `questionnaire.interests` pour les retraits.

Claviger ne dit pas :

« Retire tous les rôles non sélectionnés. »

Il dit :

« Parmi les rôles appartenant au catalogue géré par ce questionnaire, retire ceux qui ne sont plus désirés. »

Les rôles administratifs, trusted ou gérés par un autre mécanisme ne font pas partie de cet ensemble et restent donc intacts.

11.3 Idempotence pratique

Si l'utilisateur soumet exactement la sélection qu'il possède déjà :

```

Etat actuel = Etat désiré
|
v
add_role_ids = ()
remove_role_ids = ()

```

L'executor détecte alors :

```

if not plan.has_changes:
    return MemberRoleExecutionResult(...)

```

Il ne touche pas Discord.

Cette propriété est utile dans un bot : relancer `/membre` ou soumettre deux fois le même état ne doit pas multiplier les effets de bord.

12. L'Executor : la frontière où le plan devient réel

Le planner manipule des entiers. L'executor doit les transformer en objets Discord réels puis appeler l'API.

12.1 Le preflight

Avant toute mutation :

```
for role_id in planned_role_ids:
    role = guild.get_role(role_id)

    if role is None:
        raise MemberRoleNotFoundError(...)

    if role.managed or role.position >= bot_member.top_role.position:
        raise MemberRoleNotManageableError(...)

    roles_by_id[role_id] = role
```

Pourquoi valider **tous** les rôles avant de commencer ?

Imaginons le plan :

```
- rôle 10
+ rôle 20
+ rôle 30
```

Si le rôle 30 n'existe plus et que le code découvre cette erreur seulement après avoir retiré 10 et ajouté 20, le plan était invalide avant même la première opération.

Le preflight permet donc de distinguer deux types d'échec :

- **plan invalide avant exécution** : aucune mutation ne doit partir ;
- **échec réseau pendant l'exécution d'un plan valide** : une partie des mutations a éventuellement déjà été appliquée.

Le premier cas peut être rendu proprement atomique. Le second ne peut pas devenir une vraie transaction Discord et doit être observé/reporté correctement.

12.2 Pourquoi tester `role.managed`

Discord possède des rôles administrés par des intégrations ou mécanismes internes.

Même si leur position semble basse dans la hiérarchie, le bot ne doit pas supposer qu'il peut les manipuler.

`role.managed` exprime cette propriété côté API.

12.3 Pourquoi comparer les positions de rôles

Discord impose sa propre hiérarchie : un bot ne peut pas gérer un rôle situé au-dessus de son rôle principal.

Claviger vérifie :

```
role.position >= bot_member.top_role.position
```

Cette validation applicative n'annule pas les contrôles de Discord. Elle permet simplement de détecter le problème avant d'envoyer une mutation destinée à échouer.

12.4 Le rôle de `RoleManager`

`RoleManager` est volontairement petit :

```
async def add_role(self, member, role, *, reason=None) -> bool:
    if role in member.roles:
        return False

    await member.add_roles(role, reason=reason)
    return True
```

Il encapsule les appels `discord.Member.add_roles()` et `remove_roles()`.

Pourquoi ne pas appeler directement ces méthodes dans l'executor ?

Parce que ce petit adaptateur fournit un point de substitution :

- les tests d'executor peuvent mocker **RoleManager** ;
- une future stratégie de retry/reporting peut être centralisée ;
- la sémantique « ne rien faire si déjà présent/absent » est partagée.

Ce n'est pas une couche métier lourde. C'est une **frontière d'effet de bord**.

13. /noctis : même architecture, domaine plus riche

Le workflow Noctis réutilise la même idée générale mais ajoute un modèle métier spécifique.

Un thème visible par l'utilisateur correspond à :

```
thème logique
|
+--> accès de base : no-ia-<theme>
|
'--> variante IA : ia-<theme> (optionnelle)
```

L'utilisateur ne choisit pas séparément « classique X » puis « IA X ».

Il choisit :

1. les thèmes ;
2. une préférence globale `include_ai`.

13.1 Transformer des lignes techniques en thèmes métier

AdultAccessWorkflowService commence par construire un index des variantes IA :

```
ai_by_theme = {
    self._extract_theme_key(access.access_key, prefix=self.AI_PREFIX): access
    for access in entries
    if access.access_key.startswith(self.AI_PREFIX)
    and access.is_publicly_ready
}
```

Puis seules les entrées **no-ia-*** servent de base aux thèmes :

```
for access in entries:
    if not access.access_key.startswith(self.BASE_PREFIX):
        continue

    if not access.is_publicly_ready:
        continue

    theme_key = self._extract_theme_key(...)

    themes.append(
        AdultAccessTheme(
            theme_key=theme_key,
            base_access=access,
            ai_access=ai_by_theme.get(theme_key),
        )
    )
```

C'est un exemple intéressant de séparation **modèle technique / modèle présenté**.

La DB conserve des rôles réels. Le questionnaire travaille avec un concept utilisateur : le thème.

13.2 Pourquoi ne pas stocker une colonne `is_ai`

La relation IA/non-IA est actuellement dérivable depuis la clé métier : **no-ia-X** et **ia-X**.

Le workflow reconstruit cette relation au moment où il en a besoin.

Ajouter une colonne ne serait utile que si la convention de clé ne suffisait plus ou si la sémantique devait devenir configurable indépendamment.

Dans la V1, ne pas dupliquer cette information évite d'avoir deux sources capables de se contredire.

13.3 La préférence IA est additive

La résolution des rôles fait :

```
role_ids.append(theme.base_access.role_id)

if include_ai and theme.ai_access is not None:
    role_ids.append(theme.ai_access.role_id)
```

Donc l'option IA **ajoute** la variante ; elle ne remplace pas la base.

Cette règle est ensuite reflétée par le planner Noctis, qui gère également le rôle global d'option IA et le rôle adulte.

14. Les models : transporter un état sans cacher des effets de bord

La majorité des models sont des `dataclass`.

Exemple :

```
@dataclass(frozen=True, slots=True)
class MemberRolePlan:
    add_role_ids: tuple[int, ...] = ()
    remove_role_ids: tuple[int, ...] = ()
```

14.1 Pourquoi `dataclass`

Une `dataclass` est adaptée lorsqu'un objet sert surtout à transporter des données structurées.

Python génère notamment automatiquement un constructeur et des méthodes de représentation/comparaison utiles.

Cela évite du boilerplate sans transformer le model en objet magique.

14.2 Pourquoi `frozen=True`

Un model frozen ne peut pas être muté normalement après création.

Pour un plan, c'est particulièrement logique :

```
planner produit le plan
      |
      v
le plan est un résultat
      |
      v
executor le consomme
```

Si l'executor modifiait silencieusement le plan qu'il reçoit, le débogage deviendrait beaucoup plus difficile.

14.3 Pourquoi `slots=True`

Les slots limitent la création dynamique d'attributs et réduisent le poids de petits objets nombreux.

Dans Clavier, le bénéfice mémoire est secondaire. Le bénéfice pédagogique est surtout de rendre les objets plus stricts : une faute de frappe créant accidentellement un nouvel attribut n'est pas acceptée aussi facilement.

14.4 Pourquoi des `tuple` plutôt que des `list` dans beaucoup de résultats

Une `list` exprime naturellement une collection modifiable.

Un `tuple` exprime mieux :

« Voici le résultat calculé ; il n'est pas destiné à être enrichi après sa construction. »

Cette convention renforce l'idée des models immuables.

14.5 Les propriétés calculées

```
@property
def change_count(self) -> int:
    return len(self.add_role_ids) + len(self.remove_role_ids)
```

`change_count` n'est pas stocké. Il est dérivé des données déjà présentes.

C'est préférable à un troisième champ qu'il faudrait maintenir en cohérence manuellement.

15. SQLite : pourquoi une base locale convient à la V1

Claviger utilise SQLite pour stocker :

- les polices de guild ;
- le catalogue des intérêts ;
- le catalogue des accès adultes ;
- les métadonnées humaines et l'état de synchronisation.

Pour une instance de bot à faible volume, SQLite apporte :

- aucune infrastructure serveur supplémentaire ;
- un fichier facile à sauvegarder ;
- des transactions réelles ;
- des contraintes SQL ;
- une migration possible vers une DB serveur plus tard si le besoin apparaît.

La décision importante n'est pas seulement « SQLite est simple ». C'est que la charge actuelle n'exige pas un SGBD réseau juste pour avoir l'air plus professionnel.

16. DatabaseConnection : ouvrir proprement une connexion asynchrone

Le coeur du wrapper est :

```
@asynccontextmanager
async def connect(self):
    connection = await aiomysql.connect(self.database_path)

    try:
        await connection.execute("PRAGMA foreign_keys = ON")
        yield connection
    finally:
        await connection.close()
```

16.1 Pourquoi @asynccontextmanager

Cela permet d'écrire :

```
async with self.database.connect() as connection:
    ...
```

Le contexte garantit que la connexion passe par le **finally** même si le bloc lève une exception.

C'est l'équivalent asynchrone du principe :

```
ouvrir ressource
try:
    travailler
finally:
    fermer ressource
```

Le gain est important pour une DB : une exception métier ne doit pas laisser des connexions ouvertes parce qu'un `close()` a été oublié dans un chemin d'erreur.

16.2 Pourquoi activer PRAGMA foreign_keys = ON à chaque connexion

Dans SQLite, l'application des foreign keys est une propriété de connexion.

La mettre dans le wrapper central évite de dépendre de la discipline de chaque repository.

Toute connexion ouverte par Claviger reçoit la même règle.

16.3 Pourquoi Path

```
self.database_path = Path(database_path)
```

`pathlib.Path` représente un chemin de fichier comme un objet plutôt que comme une chaîne brute.

Le reste du code peut alors utiliser `is_file()`, `.parent.mkdir(...)`, etc., avec une API portable et lisible.

17. Les repositories : le SQL derrière une frontière stable

Un repository répond à une question du domaine de persistance, par exemple :

```
« donne-moi les intérêts du guild 123 »
```

et non :

```
« exécute ce SELECT écrit par le caller »
```

Le service n'a pas besoin de connaître le nom de table ou l'ordre des colonnes.

17.1 Pourquoi `aiosqlite.Row`

```
connection.row_factory = aiosqlite.Row
```

Par défaut, SQLite renvoie des tuples positionnels.

Avec `Row`, le repository peut convertir une ligne en model en utilisant des noms de colonnes. C'est plus robuste et lisible qu'une dépendance permanente à « l'index 7 correspond à telle propriété ».

17.2 Paramètres SQL et identifiants SQL : deux problèmes différents

Les valeurs utilisateur ou runtime sont passées comme paramètres :

```
WHERE guild_id = ?
AND role_id = ?
```

avec :

```
(guild_id, role_id)
```

C'est la manière correcte de transmettre des **valeurs** à SQLite.

En revanche, un nom de table ou de colonne ne peut pas être bindé avec `?`. Le repository générique doit interpoler :

```
FROM {self.table_name}
```

C'est pourquoi il valide ces identifiants contre une expression régulière stricte lors de la construction :

```
_IDENTIFIER_PATTERN = re.compile(r"^[A-Za-z_][A-Za-z0-9_]*$")
```

La leçon générale est :

- les **valeurs SQL** doivent être paramétrées ;
- les **identifiants dynamiques** doivent provenir d'une configuration contrôlée et être validés.

17.3 Pourquoi un repository générique commun

`RoleChannelCatalogRepository` contient le comportement commun aux catalogues rôle -> salon.

`InterestCatalogRepository` et `AccessCatalogRepository` peuvent ensuite fournir :

- le nom de table ;
- le nom de colonne clé ;
- le type de model ;
- l'exception métier spécifique.

Cela évite deux repositories massivement copiés tout en conservant des noms métier clairs à l'extérieur.

`Generic` et `TypeVar` servent ici surtout au typage : le repository générique peut promettre de renvoyer le bon sous-type d'entrée catalogue.

18. Schéma et migrations : faire évoluer un fichier SQLite existant

DatabaseSchema utilise :

```
CURRENT_SCHEMA_VERSION = 4
```

et un dictionnaire :

```
MIGRATIONS: dict[int, Sequence[str]] = {...}
```

18.1 PRAGMA user_version

SQLite fournit un entier libre nommé `user_version`.

Claviger s'en sert comme version applicative du schéma :

```
cursor = await connection.execute("PRAGMA user_version")
```

Après une migration :

```
await connection.execute(f"PRAGMA user_version = {version}")
```

Le fichier DB sait donc à quelle génération du schéma il appartient.

18.2 Pourquoi séparer `initialize()` et `migrate()`

Une DB inexistante/non initialisée et une DB ancienne sont deux états différents.

`initialize()` refuse une DB déjà versionnée.

`migrate()` refuse une DB version 0.

Cette séparation empêche une commande d'administration de faire implicitement autre chose que ce qu'elle annonce.

18.3 Pourquoi `BEGIN IMMEDIATE`

Pour une migration :

```
await connection.execute("BEGIN IMMEDIATE")
...
await connection.commit()
```

En cas d'échec :

```
await connection.rollback()
```

Le but est simple : ne pas laisser une migration à moitié appliquée.

Le lock d'écriture est acquis dès le début de la transaction, ce qui rend l'intention d'écriture explicite et limite les surprises de concurrence sur le fichier.

19. DatabaseStatusService : diagnostiquer sans modifier

La V1 distingue volontairement :

```
observer l'état de la DB
```

et :

```
initialiser / migrer la DB
```

`DatabaseStatusService.check()` retourne un `DatabaseStatus` parmi :

```
MISSING
UNINITIALIZED
READY
MIGRATION_REQUIRED
TOO_NEW
UNAVAILABLE
```

Cette modélisation rend les commandes plus faciles à comprendre : `/membre` a seulement besoin de savoir si l'état est `READY`.

Les commandes d'administration peuvent, elles, expliquer précisément pourquoi la DB ne l'est pas.

Le service de statut ne tente pas de « réparer » silencieusement l'environnement.

20. Le catalogue : une copie contrôlée de l'état Discord

La base ne remplace pas Discord comme source de vérité sur l'existence réelle des rôles et salons.

Elle sert plutôt de **catalogue enrichi** :

```
identité Discord stable
+
snapshot du nom
+
mapping salon
+
clé métier
+
métadonnées humaines
+
état de validité/synchronisation
```

Un `RoleChannelCatalogEntry` contient donc à la fois :

- `role_id`, `channel_id` : identités stables Discord ;
- `role_name`, `channel_name` : snapshots lisibles ;
- `catalog_key` : identité métier ;
- `label`, `description`, `emoji` : présentation ;
- des booléens décrivant la santé du mapping.

Cela explique pourquoi supprimer une ligne dès qu'un rôle disparaît serait une perte d'information : le catalogue doit pouvoir dire « cette entrée connue n'est actuellement plus présente ».

21. La synchronisation de catalogue : observer, comparer, appliquer

Le catalog sync est une bonne miniature de l'architecture générale de Clavier.

```
Discord réel
|
v
RoleChannelDiscoveryService
|
v
GuildRoleChannelSnapshot
|
v
CatalogSyncPlanner
|
v
CatalogSyncPlan
|
v
CatalogSyncCoordinatorService
|
v
Repositories + transaction SQLite
```

Il y a volontairement une frontière entre **observation**, **décision** et **écriture**.

21.1 RoleChannelDiscoveryService : photographier Discord

Le service parcourt les salons compatibles :

```
content_channels = tuple(
    channel
    for channel in guild.channels
    if isinstance(channel, (discord.TextChannel, discord.ForumChannel))
)
```

Puis il photographie chaque rôle :

```
DiscordRoleSnapshot(
    role_id=role.id,
    role_name=role.name,
    role_manageable=role < bot_member.top_role,
    explicit_channel_ids=tuple(
        channel.id
        for channel in content_channels
        if channel.overwrites_for(role).view_channel is True
    ),
)
```

Le mot important est **snapshot**.

Le planner suivant ne doit pas avoir besoin de rappeler Discord pour chaque comparaison. Il reçoit une représentation figée de l'état observé à cet instant.

Cela apporte aussi de la testabilité : un test du planner peut fabriquer un `GuildRoleChannelSnapshot` sans serveur Discord réel.

21.2 Pourquoi regarder `overwrites_for(role).view_channel is True`

La synchronisation cherche ici un mapping **explicite** rôle -> salon.

`channel.overwrites_for(role)` retourne les permissions spécifiques configurées pour ce rôle sur le salon. En lisant `view_channel is True`, le service cherche les salons où la permission de visibilité a été explicitement autorisée pour le rôle.

Le but n'est pas de refaire tout le moteur de permissions Discord. Le but est de reconnaître la convention utilisée par le serveur pour associer un rôle à un espace.

21.3 Pourquoi le planner exige un mapping unique

```
if len(role.explicit_channel_ids) != 1:
    return None
```

Pour un catalogue rôle -> salon, zéro mapping est incomplet et plusieurs mappings sont ambigus.

Le planner refuse donc de deviner.

Il produit un warning :

```
missing_channel_mapping
```

ou :

```
ambiguous_channel_mapping
```

C'est une application très concrète du principe **fail closed / don't guess** : lorsque la convention Discord n'est pas suffisamment claire, le catalogue n'invente pas la relation.

21.4 Le planner produit plusieurs catégories d'action

`CatalogSyncPlan` sépare conceptuellement :

- les nouvelles entrées à créer ;
- les entrées existantes dont les données Discord ont changé ;
- les changements d'état d'entrées devenues invalides/incomplètes ;
- les warnings nécessitant une attention humaine.

Cette distinction évite de traiter « rôle renommé », « rôle supprimé » et « mapping ambigu » comme le même cas.

21.5 Pourquoi un même snapshot sert à tous les catalogues

Dans le coordinator :

```
snapshot = self.discovery_service.build_snapshot(guild)
definitions = self.registry.for_policy(policy)

for definition in definitions:
    current_entries = await definition.repository.list_for_guild(guild.id)
    plan = self.planner.build_plan(
```

```

        current_entries,
        snapshot,
        prefix=definition.prefix,
    )

```

Interest et Access sont deux lectures différentes du même serveur Discord.

Faire deux discoveries réseau indépendantes pourrait produire deux visions légèrement différentes si la configuration change entre les deux. Un snapshot commun garantit que les deux plans ont été calculés à partir du même instant logique.

22. CatalogRegistry : mutualiser l'orchestration sans fusionner les métiers

Le registry contient :

```

CatalogDefinition(
    catalog_key="member_interests",
    display_name="Member interests",
    entry_name="Member interest",
    prefix=policy.member_interest_prefix,
    repository=self.interest_repository,
)

```

et l'équivalent pour les accès adultes.

Son rôle n'est pas de stocker le catalogue lui-même.

Il répond à :

« Quels catalogues sont gérés, avec quelle convention et quel repository ? »

Grâce à lui, /claviger catalog sync et /claviger catalog next peuvent être globaux sans recopier une version Interest puis une version Access.

C'est un bon exemple de **généricité placée au bon niveau** :

- l'orchestration commune est mutualisée ;
- les repositories restent identifiables métier ;
- les règles particulières du questionnaire Noctis restent dans son workflow.

23. Pourquoi la synchronisation utilise une seule transaction

Le coordinator construit d'abord **tous** les plans. Ensuite seulement il ouvre une connexion d'écriture :

```

async with self.database.connect() as connection:
    await connection.execute("BEGIN IMMEDIATE")

    try:
        for definition, catalog_result in zip(...):
            await definition.repository.apply_sync_plan(
                connection,
                guild.id,
                catalog_result.plan,
            )

        await connection.commit()

    except Exception:
        await connection.rollback()
        raise

```

Le détail important est que les repositories reçoivent **la même connexion**.

Ils ne commitent pas indépendamment chaque catalogue.

Sans cela :

```

Interest sync -> COMMIT
Access sync   -> erreur

```

laisserait la DB dans un état où un catalogue représente le nouveau snapshot et l'autre l'ancien.

Avec la transaction globale :

```

Interest OK
Access erreur

```

```
|
v
ROLLBACK de l'ensemble
```

Cette transaction est une vraie transaction SQLite. À la différence des mutations de rôles Discord, la persistance locale peut offrir une atomicité complète.

24. Les politiques : séparer la logique générique de la configuration d'un serveur

GuildPolicy contient les règles effectives :

```
@dataclass(frozen=True)
class GuildPolicy:
    member_role_name: str
    adult_role_name: str
    member_interest_prefix: str
    adult_access_prefix: str
    salutations_channel_name: str
    adult_rules_channel_name: str
    role_management_enabled: bool
    adult_access_enabled: bool
```

La commande /**membre** ne devrait donc pas connaître directement **salutations**, et le classifieur ne devrait pas inventer le préfixe **interest**.

Ils demandent à la policy.

24.1 Pourquoi GuildPolicyOverrides contient des None

Les overrides persistés représentent uniquement ce qui est explicitement configuré.

```
member_role_name: str | None = None
```

None signifie :

« aucun override, conserver la valeur de base »

et non :

« la valeur métier est réellement absente ».

PolicyResolver._apply_overrides() construit ensuite une GuildPolicy complète.

C'est une distinction utile entre **configuration partielle stockée** et **configuration effective consommable**.

24.2 Pourquoi il existe une SAFE_DEFAULT_POLICY

Elle contient :

```
role_management_enabled=False
adult_access_enabled=False
```

Un guild inconnu ne reçoit donc pas spontanément les permissions actives d'un serveur existant.

La configuration par défaut choisit l'inaction lorsque Clavier ne peut pas prouver que le workflow doit être actif.

24.3 Le fallback spécifique de la V1

Le resolver possède également un fallback destiné au guild principal lorsque la DB est indisponible.

C'est un compromis de disponibilité propre à l'état actuel du projet : le guild explicitement configuré au bootstrap peut conserver une policy connue, alors qu'un autre guild retombe sur la policy sûre.

Cette partie fait partie des éléments à reconsidérer si le projet devient réellement multi-guild ou si les politiques deviennent plus sensibles.

25. Rôles trusted et autorisation : distinguer hiérarchie Discord et capacité applicative

RoleDiscoveryService distingue :

```
def is_trusted_role(role, bot_role) -> bool:
    return role > bot_role and not role.managed and not role.is_default()

def is_manageable_role(role, bot_role) -> bool:
    return role < bot_role and not role.managed and not role.is_default()
```

Dans la règle métier actuelle :

```
au-dessus de Claviger -> humain de confiance
Claviger
sous Claviger         -> rôle potentiellement gérable
```

Cette convention utilise directement la hiérarchie Discord comme frontière lisible par les administrateurs du serveur.

25.1 Pourquoi ajouter malgré tout la notion de Capability

```
class Capability(StrEnum):
    SAY = "say"
    SAY_PLAIN = "say.plain"
```

Une capacité représente **une action applicative**, pas un rôle Discord.

L'autorisation devient donc :

```
Utilisateur + guild + capability
                |
                v
AuthorizationService
                |
                v
True / False
```

C'est plus évolutif que d'écrire partout :

```
if "Admin" in [role.name for role in member.roles]:
```

Le mécanisme peut évoluer sans changer la signature conceptuelle des commandes.

25.2 Pourquoi /say est un bon exemple

La commande demande d'abord :

```
Capability.SAY
```

puis, pour le mode plain :

```
Capability.SAY_PLAIN
```

La seconde capacité est plus sensible car un message plain ressemble davantage à un message natif du bot sans signature visuelle par embed.

Dans la V1, SAY_PLAIN n'est pas présent dans les capacités trusted par défaut ; le propriétaire du serveur reste autorisé car il est traité explicitement en premier.

Le code exprime ainsi une idée utile : **une même commande peut contenir plusieurs niveaux de pouvoir**.

26. SayService : un exemple simple de boundary Discord réutilisable

La commande /say valide :

- la guild ;
- l'autorisation ;
- l'appartenance du salon au même serveur ;
- la capacité supplémentaire en mode plain.

Puis elle délègue l'effet :

```
await say_service.send(
    salon,
    message,
    style=say_style,
    color=interaction.user.color,
)
```

Le service transforme ensuite le style métier en primitive Discord :

```
if style is SayStyle.PLAIN:
    await channel.send(message)
    return

embed = discord.Embed(
    description=message,
    color=color,
)

await channel.send(embed=embed)
```

`channel.send()` est la méthode `discord.py` qui effectue l'envoi effectif dans un salon textuel.

`discord.Embed` construit une représentation riche que Discord rend dans le client : couleur latérale, description, champs éventuels, footer, timestamp, etc.

La commande décide **qui peut demander l'action**. Le service décide **comment l'action est rendue**.

27. Les groupes de commandes administratives

Claviger expose un groupe racine :

```
claviger_group = app_commands.Group(
    name="claviger",
    description="Commandes d'administration de Claviger.",
)
```

Puis ajoute des sous-groupes :

```
claviger_group.add_command(roles_group)
claviger_group.add_command/catalog_group)
claviger_group.add_command(report_group)
claviger_group.add_command(database_group)
claviger_group.add_command(guild_group)
```

Cela produit conceptuellement :

```
/claviger roles ...
/claviger catalog ...
/claviger report ...
/claviger database ...
/claviger guild ...
```

Pourquoi ne pas tout enregistrer à plat ?

Parce qu'une commande d'administration a deux identités :

- la famille fonctionnelle (catalog, database, etc.) ;
- l'opération (sync, status, migrate, etc.).

Le groupe reflète cette hiérarchie dans l'UX Discord et dans le code.

28. Le reporting : un événement, plusieurs destinations

La V1 possède déjà une architecture de reporting séparée de la destination.

Le coeur est :

```
@dataclass(frozen=True)
class ReportEvent:
    event_type: str
    severity: ReportSeverity
    title: str
    summary: str
    details: str | None = None
    guild_id: int | None = None
    actor_id: int | None = None
```

```
target_id: int | None = None
...
```

Un événement décrit **ce qui s'est passé**.

Il ne sait pas s'il sera affiché dans un forum Discord ou écrit dans un logger Python.

28.1 Pourquoi Reporter est un Protocol

```
class Reporter(Protocol):
    async def report(self, event: ReportEvent) -> None:
        ...
```

Un **Protocol** décrit ici une forme attendue : n'importe quel objet possédant la bonne méthode asynchrone peut servir de reporter.

Il n'est pas nécessaire d'imposer une grosse classe abstraite avec héritage.

Cela permet à :

```
PythonLoggingReporter
DiscordForumReporter
```

de satisfaire le même contrat.

Une destination future (Sentry, webhook, fichier JSON, etc.) pourrait s'insérer au même endroit si le besoin apparaît.

28.2 ReportService fait du fan-out

```
for reporter in self.reporters:
    try:
        await reporter.report(event)
    except Exception:
        self.logger.exception(...)
```

Le service envoie le même événement à toutes les destinations configurées.

Surtout, l'échec d'un reporter n'empêche pas les suivants de recevoir l'événement.

Exemple : si le forum Discord de reporting est mal configuré, le logger Python doit encore pouvoir conserver une trace.

28.3 Pourquoi DiscordForumReporter reçoit le discord.Client

Il doit résoudre un channel par ID :

```
channel = self.client.get_channel(self.forum_channel_id)

if channel is None:
    channel = await self.client.fetch_channel(self.forum_channel_id)
```

`get_channel()` consulte le cache local du client.

`fetch_channel()` effectue une requête API si l'objet n'est pas disponible dans le cache.

Cette stratégie « cache puis réseau » évite une requête inutile lorsque le client connaît déjà le forum.

28.4 Pourquoi créer un thread de forum

```
await forum.create_thread(
    name=thread_name,
    embed=embed,
)
```

Un forum Discord organise naturellement les incidents comme des sujets séparés plutôt que comme une longue succession de messages dans un salon classique.

Le reporter convertit `ReportEvent` en `discord.Embed` puis crée le post.

Le service métier n'a aucune connaissance de cette représentation.

28.5 Etat V1 : distinction entre bonne architecture et branchement incomplet

Le design `ReportEvent -> ReportService -> Reporter` est déjà une bonne frontière.

En revanche, l'audit a identifié que toutes les boundaries publiques ne l'utilisent pas encore de manière uniforme et que plusieurs erreurs ne conservent pas suffisamment de contexte/traceback.

Il faut donc comprendre le reporting comme une **infrastructure déjà posée mais encore à généraliser**, pas comme un système d'observabilité terminé.

29. Les exceptions : donner un sens à l'échec

Plusieurs services définissent des exceptions spécifiques :

```
class MemberWorkflowError(RuntimeError): ...
class MemberRoleManagementDisabledError(MemberWorkflowError): ...
class MemberBaseRoleNotFoundError(MemberWorkflowError): ...
```

ou :

```
class MemberRoleNotFoundError(MemberRoleExecutionError): ...
class MemberRoleNotManageableError(MemberRoleExecutionError): ...
```

Pourquoi ne pas retourner simplement `False` ?

Parce qu'un échec de workflow peut avoir plusieurs significations :

- fonctionnalité désactivée ;
- rôle requis absent ;
- rôle planifié disparu ;
- rôle non gérable ;
- DB indisponible ;
- API Discord en erreur.

Une exception typée permet à une couche supérieure de choisir la bonne réponse ou le bon niveau de log.

La V1 mélange encore parfois ces exceptions sous un `except Exception` au niveau des handlers. Le chantier de logging V1.1 doit précisément améliorer cette dernière étape sans supprimer la typologie utile déjà présente.

30. Quelques mécanismes Python qui structurent réellement le projet

Cette section ne cherche pas à enseigner tout Python. Elle explique les constructions que l'on rencontre souvent en lisant Clavier.

30.1 StrEnum

Exemple :

```
class DatabaseState(StrEnum):
    READY = "ready"
    UNAVAILABLE = "unavailable"
```

Un enum évite de disperser des chaînes libres telles que `"ready"` dans tout le programme.

Avec `StrEnum`, les valeurs restent naturellement sérialisables/affichables comme chaînes tout en bénéficiant d'une identité typée.

30.2 set

Les planners utilisent beaucoup les ensembles :

```
selected_keys = set(selected_interest_keys)
known_keys = {interest.catalog_key for interest in questionnaire.interests}
unknown_keys = selected_keys - known_keys
```

Le `set` est approprié pour :

- tester rapidement l'appartenance ;
- supprimer les doublons ;
- calculer une différence d'ensembles.

Ici, `selected_keys - known_keys` se lit presque comme la règle métier : « les sélections inconnues sont les clés demandées qui ne font pas partie des clés connues ».

30.3 Générateurs et itérables

Noctis passe :

```
member_role_ids=(role.id for role in member.roles)
```

Le planner annonce accepter un `Iterable[int]` plutôt qu'exiger une liste précise.

Le caller peut donc fournir une liste, un tuple, un set ou un générateur tant que le service n'exige pas une caractéristique plus spécifique.

30.4 `dict.fromkeys(...)` pour dédupliquer en conservant l'ordre

Les executors utilisent :

```
planned_role_ids = tuple(
    dict.fromkeys(plan.remove_role_ids + plan.add_role_ids)
)
```

Un dictionnaire moderne conserve l'ordre d'insertion. `fromkeys` sert ici à supprimer un éventuel doublon d'ID tout en gardant un ordre déterministe.

30.5 `zip(..., strict=True)`

Dans le catalog sync :

```
for definition, catalog_result in zip(
    definitions,
    result.catalogs,
    strict=True,
):
```

`strict=True` demande à Python de lever une erreur si les deux séquences n'ont pas la même longueur.

Sans cela, `zip` s'arrête silencieusement à la plus courte. Dans une transaction de synchronisation, une divergence structurelle ne doit pas passer inaperçue.

30.6 `raise ... from error`

Exemple DB :

```
except aiosqlite.Error as error:
    raise DatabaseUnavailableError(...) from error
```

Le code remonte une exception qui parle le langage de Clavier **sans perdre la cause technique originale**.

La chaîne d'exception devient exploitable dans un traceback : erreur applicative en haut, cause SQLite en dessous.

31. Les tests comme documentation exécutable

La suite de tests est volumineuse, mais son intérêt pédagogique n'est pas le nombre brut. Les tests montrent quelles garanties le projet considère importantes.

31.1 Tester un executor sans Discord réel

Le test construit des `Mock(spec=discord.Role)` et un `RoleManager` dont les méthodes sont des `AsyncMock`.

Puis il vérifie notamment :

```
role_manager.add_role.assert_not_awaited()
role_manager.remove_role.assert_not_awaited()
```

après un plan vide.

Cela documente une garantie : **un plan sans changement ne doit produire aucun appel Discord**.

31.2 Le test du preflight décrit une règle d'architecture

Un autre test fournit :

```
rôle à retirer valide
+
rôle à ajouter inexistant
```

et vérifie que **ni remove ni add n'ont été appelés**.

Ce test ne vérifie pas une ligne de code arbitraire. Il capture la règle :

« *un plan invalide doit échouer avant la première mutation* ».

Quand un futur refactoring casse cette propriété, le test sert de garde-fou architectural.

31.3 pytest.mark.asyncio

Les tests marqués :

```
@pytest.mark.asyncio
async def test_...():
```

peuvent **await** les services asynchrones.

Sans plugin asynchrone, pytest ne saurait pas exécuter directement ces coroutines comme des tests ordinaires.

31.4 Pourquoi utiliser spec=discord.Member

```
member = Mock(spec=discord.Member)
```

Le **spec** limite le mock aux attributs/méthodes correspondant à l'objet réel.

Cela ne remplace pas un test d'intégration Discord, mais évite certains mocks trop permissifs où une faute de nom d'attribut passerait malgré tout.

31.5 Les tests DB utilisent un vrai fichier temporaire

Les tests de repository utilisent **tmp_path** :

```
database = DatabaseConnection(tmp_path / "claviger.db")
schema = DatabaseSchema(database)
await schema.initialize()
```

Ici, mocker SQLite apporterait peu de valeur. On teste précisément la couche qui parle à SQLite, donc un fichier temporaire réel est plus pertinent.

C'est la différence entre :

- **test unitaire du planner** : aucune infrastructure ;
- **test d'intégration du repository** : vraie DB SQLite isolée ;
- **smoke test Discord** : vraie plateforme externe.

32. Comment diagnostiquer Claviger sans relire 77 fichiers

Voici une carte de dépannage orientée symptômes.

32.1 /membre n'apparaît pas dans Discord

Commencer par :

```
main.py
-> bot.py
-> tree.add_command(create_member_command(...))
-> setup_hook()
-> tree.sync(guild=...)
```

Vérifier ensuite :

- **DISCORD_GUILD_ID** ;

- que le bot démarre réellement ;
- le résultat de la synchronisation des commandes ;
- les permissions/installation de l'application côté Discord.

Ne pas commencer par le planner : il n'intervient même pas encore.

32.2 /membre apparaît mais refuse immédiatement

Lire `commands/member_command.py` dans l'ordre.

Les refus avant ouverture viennent essentiellement de :

- contexte hors guild ;
- DB non **READY** ;
- policy impossible à charger ;
- gestion des rôles désactivée ;
- mauvais salon ;
- échec de construction du questionnaire.

32.3 Le questionnaire s'ouvre mais une option manque

Chemin probable :

```
MemberInterestQuestionnaireService
-> InterestCatalogRepository
-> RoleChannelCatalogEntry.is_publicly_ready / filtres
```

Puis, si l'entrée DB elle-même est mauvaise :

```
/claviger catalog sync
-> RoleChannelDiscoveryService
-> CatalogSyncPlanner
-> repository
```

Questions à poser :

- l'entrée existe-t-elle en DB ?
- est-elle **enabled** ?
- label/description sont-ils renseignés ?
- le rôle existe-t-il encore ?
- est-il considéré gérable ?
- le salon associé existe-t-il ?
- le mapping est-il unique ?
- le rôle correspond-il encore à la policy/prefix ?

32.4 Les bonnes cases sont visibles mais une sélection produit les mauvais rôles

Commencer par le planner :

```
MemberRolePlannerService
```

ou :

```
NoctisRolePlannerService
-> AdultAccessWorkflowService
```

Construire mentalement :

```
current_role_ids
selected_keys
desired_role_ids
add_role_ids
remove_role_ids
```

Si le plan calculé est déjà faux, l'executor n'est pas la cause.

32.5 Le plan est bon mais Discord ne change rien

Lire :

```
...RoleExecutorService
-> RoleManager
-> member.add_roles/remove_roles
```

Vérifier :

- rôle toujours présent ;
- `role.managed` ;
- position par rapport au rôle de Claviger ;
- permissions du bot ;
- exception réseau/API ;
- état partiellement appliqué en cas d'échec en cours de séquence.

32.6 /noctis affiche le mauvais état IA

Commencer par :

```
include_ai = any(role.name == AI_OPTION_ROLE_NAME for role in member.roles)
```

puis vérifier le rôle global d'option IA et le modèle de pairing `no-ia-*` / `ia-*` dans `AdultAccessWorkflowService`.

32.7 /claviger catalog sync ne découvre pas le mapping attendu

Commencer par `RoleChannelDiscoveryService`.

Le service ne cherche que les `TextChannel` et `ForumChannel`, puis uniquement les overwrites où `view_channel` is `True` explicitement.

Si le rôle voit un salon uniquement par héritage d'une autre permission, cela ne correspond pas nécessairement à la convention de discovery.

Ensuite inspecter `CatalogSyncPlanner._resolve_unique_channel()` : un rôle doit aboutir à **exactement un** salon explicite pour un mapping valide.

32.8 Une commande fonctionne mais aucun rapport n'arrive dans le forum

Suivre :

```
commande
-> ReportService.emit
-> PythonLoggingReporter
-> DiscordForumReporter
-> get_channel / fetch_channel
-> ForumChannel.create_thread
```

Vérifier également `ADMIN_REPORT_FORUM_ID`.

Comme les reporters sont isolés les uns des autres, une panne du forum ne devrait pas empêcher le reporter Python de s'exécuter.

33. Où placer une modification : guide de décision

Cette section ne dicte pas une architecture théorique. Elle aide à choisir le bon endroit dans l'architecture actuelle.

33.1 « Je change le texte ou le format Discord »

Probablement :

```
commands/
ui/
reporting/discord_forum.py
```

Exemples : titre de modal, libellé, embed, message éphémère.

33.2 « Je change la règle qui décide quels rôles doivent exister »

Probablement :

```
...planner_service.py
AdultAccessWorkflowService
```

Le test principal devrait pouvoir s'écrire sans réseau.

33.3 « Je change la manière d'appliquer un rôle »

Probablement :

```
...executor_service.py
RoleManager
```

Le planner ne doit pas connaître ce détail.

33.4 « Je change ce qu'une entrée de catalogue doit contenir »

Cela peut traverser plusieurs couches :

```
schema migration
-> model
-> repository
-> sync planner si donnée Discord
-> UI/service si donnée consommée
-> tests
```

Une modification persistante est rarement « juste un champ à ajouter ».

33.5 « J'ajoute un nouveau catalogue similaire »

Examiner :

```
CatalogRegistry
RoleChannelCatalogRepository
CatalogSyncPlanner
CatalogSyncCoordinatorService
```

Puis décider si le nouveau catalogue suit réellement le même contrat rôle -> salon. S'il ne le fait pas, ne pas forcer sa logique dans l'abstraction existante juste pour réutiliser du code.

33.6 « J'ajoute une nouvelle slash command »

Chemin recommandé dans l'architecture actuelle :

```
1. définir le cas d'usage/service
2. écrire les tests du comportement
3. créer la factory de commande
4. injecter les dépendances depuis bot.py
5. enregistrer la commande dans la CommandTree ou le bon Group
6. tester l'Interaction Discord réelle
```

La commande doit rester le traducteur entre Discord et le service, pas devenir le service elle-même.

33.7 « J'ajoute une destination de reporting »

Le point d'extension est **Reporter**.

Créer un objet possédant :

```
async def report(self, event: ReportEvent) -> None:
    ...
```

puis l'ajouter à la composition dans **bot.py**.

Les commandes ne devraient pas être modifiées pour connaître cette nouvelle destination.

34. Comment raisonner sur une future interface Web ou IA

La V1 ne contient pas encore d'agent IA ni d'interface Web. La bonne lecture du code actuel est cependant de repérer ce qui est **réutilisable** et ce qui est **Discord-spécifique**.

34.1 Réutilisable ou presque

- planners de rôles ;

- models de plans/résultats ;
- repositories ;
- catalog sync planner ;
- policies ;
- certains coordinators après abstraction de leurs paramètres Discord ;
- ReportEvent comme format d'événement.

34.2 Spécifique à Discord

- discord.Interaction ;
- discord.ui.Modal ;
- CheckboxGroup ;
- TextChannel.send() ;
- Member.add_roles() ;
- découverte de hiérarchie et overwrites ;
- forum reporter.

La future frontière saine ressemble donc à :

```
Discord command ----+
Web endpoint -----+--> application capability --> service métier
AI tool -----+
```

L'agent ne devrait pas apprendre à modifier SQLite ou appeler `member.add_roles()` directement.

Il devrait invoquer une capacité applicative disposant des mêmes contrôles que les autres consommateurs.

Pour les opérations sensibles, l'autorisation d'un humain et l'autonomie accordée à l'IA sont deux problèmes différents : une action techniquement disponible à un utilisateur trusted ne doit pas devenir automatiquement une action autonome de l'agent.

35. Les limites actuelles à garder en tête pendant la lecture

Ce guide explique l'état fourni ; il ne transforme pas les recommandations d'audit en fonctionnalités déjà présentes.

Quelques points ont une importance particulière pour le maintien :

1. plusieurs boundaries attrapent encore **Exception** de façon large ;
2. le reporting n'est pas encore uniforme sur tous les parcours ;
3. les erreurs remontées sous forme de **str(error)** ne conservent pas toujours la stack trace utile ;
4. une séquence de mutations Discord peut échouer après quelques opérations déjà réalisées ;
5. la découverte **role_manageable** doit être alignée avec les critères des executors ;
6. la nomenclature de fichiers et l'organisation de certains tests sont encore en cours de normalisation.

Ces points ne changent pas la carte mentale de l'architecture. Ils indiquent surtout où la V1.1 doit renforcer l'exploitation.

36. Parcours de lecture conseillé dans le code réel

Pour s'imprégner du projet sans lire les 6 000+ lignes source dans l'ordre, suivre ce parcours.

Etape 1 - 10 minutes : démarrage et composition

Lire :

```
pyproject.toml
src/claviger/main.py
src/claviger/bot.py
```

Objectif : être capable de dessiner le graphe DB -> repositories -> services -> commands.

Etape 2 - 20 minutes : /membre

Lire dans cet ordre :

```
commands/member_command.py
ui/member_questionnaire_modal.py
services/member_workflow_coordinator_service.py
services/member_interest_questionnaire_service.py
services/member_role_planner_service.py
services/member_role_executor_service.py
services/role_manager_service.py
```

Objectif : savoir expliquer à quel moment on observe, planifie puis modifie Discord.

Etape 3 - 15 minutes : modèles

Lire :

```
models/member_interest_questionnaire_model.py
models/member_role_plan_model.py
models/member_role_execution_result_model.py
models/role_channel_catalog_model.py
```

Objectif : distinguer état du questionnaire, plan désiré et résultat réellement appliqué.

Etape 4 - 20 minutes : /noctis

Lire :

```
services/adult_access_workflow_service.py
services/adult_access_questionnaire_service.py
services/noctis_workflow_coordinator_service.py
services/noctis_role_planner_service.py
services/noctis_role_executor_service.py
ui/noctis_questionnaire_modal.py
```

Objectif : comprendre le pairing **no-ia** / **ia** et la préférence additive.

Etape 5 - 30 minutes : persistance et synchronisation

Lire :

```
database/connection.py
database/schema.py
repositories/role_channel_catalog_repository.py
services/role_channel_discovery_service.py
services/catalog_sync_planner_service.py
services/catalog_sync_coordinator_service.py
services/catalog_registry_service.py
```

Objectif : comprendre ce qui vient de Discord, ce qui est conservé en DB et où se trouve la transaction.

Etape 6 - 15 minutes : contrôle et exploitation

Lire :

```
policies/*
services/authorization.py
reporting/*
```

Objectif : comprendre comment le contexte serveur et les permissions encadrent les cas d'usage.

Etape 7 - revenir aux tests

Pour chaque service important, lire un ou deux tests représentant une règle métier.

Ne pas chercher à lire toute la suite d'un bloc. Utiliser les tests comme une **loupe comportementale** sur le composant que l'on vient d'étudier.

37. Exercices de compréhension sans modifier le code

Ces exercices permettent de vérifier que le modèle mental est acquis.

Exercice A - prédire un plan /membre

Etat actuel :

```
Membre
interest-musicae
interest-codex
role-externe
```

Nouvelle sélection :

```
musicae
ludus
```

Question : quels IDs doivent apparaître dans `add_role_ids` et `remove_role_ids` ?

Réponse conceptuelle :

```
+ interest-ludus
- interest-codex
role-externe : intouché
Membre : déjà présent, donc aucun changement
```

Si cette réponse paraît naturelle, la logique de réconciliation est comprise.

Exercice B - raisonner sur une modal périmée

Un utilisateur ouvre `/membre`. Puis un administrateur désactive une entrée de catalogue avant qu'il ne soumette.

Question : pourquoi le coordinator reconstruit-il le questionnaire au submit ?

Réponse : pour que la clé envoyée par l'ancienne UI soit validée contre le catalogue actuel. Le client Discord n'est pas la source d'autorité.

Exercice C - catalog sync

Un rôle `interest-x` existe et possède un `overwrite` explicite `view_channel=True` sur deux salons.

Question : le planner doit-il choisir le premier ?

Réponse : non. Le mapping est ambigu ; le plan produit un warning et ne devine pas.

Exercice D - transaction

Le plan Interest s'applique correctement en DB, puis le repository Access lève une erreur avant le commit global.

Question : que doit contenir la DB ensuite ?

Réponse : aucun des deux nouveaux états ; le coordinator rollback la transaction commune.

38. Glossaire de lecture du projet

Adaptateur

Code qui traduit entre Clavier et un système externe. Une commande Discord ou `RoleManager` en sont des exemples.

Boundary

Frontière où l'application reçoit ou émet quelque chose vers l'extérieur : Interaction Discord, DB, logging, etc.

Capability

Action applicative nommée pouvant faire l'objet d'une décision d'autorisation.

Catalogue

Représentation persistée et enrichie des rôles Discord que Clavier sait proposer/gérer pour un workflow.

Composition root

Endroit où les implémentations concrètes sont instanciées et reliées entre elles. Dans la V1 : principalement `bot.py`.

Coordinator

Service qui orchestre plusieurs étapes d'un cas d'usage sans nécessairement porter le calcul détaillé lui-même.

Executor

Composant responsable d'appliquer les effets de bord d'un plan déjà calculé.

Guild

Un serveur Discord dans le vocabulaire de l'API.

Interaction

Objet reçu par `discord.py` lorsqu'un utilisateur invoque une slash command, soumet une modal ou déclenche une interaction similaire.

Idempotence

Propriété selon laquelle répéter une demande représentant le même état ne crée pas de mutation supplémentaire inutile.

Model

Objet de données structuré utilisé pour transporter un état ou un résultat.

Planner

Composant qui calcule ce qui doit changer sans provoquer lui-même l'effet de bord.

Policy

Configuration effective et règles de comportement propres à une guild.

Preflight

Validation de l'ensemble d'un plan avant la première mutation externe.

Repository

Frontière d'accès à la persistance qui cache les détails SQL aux services appelants.

Snapshot

Photographie immuable d'un état observé à un instant donné.

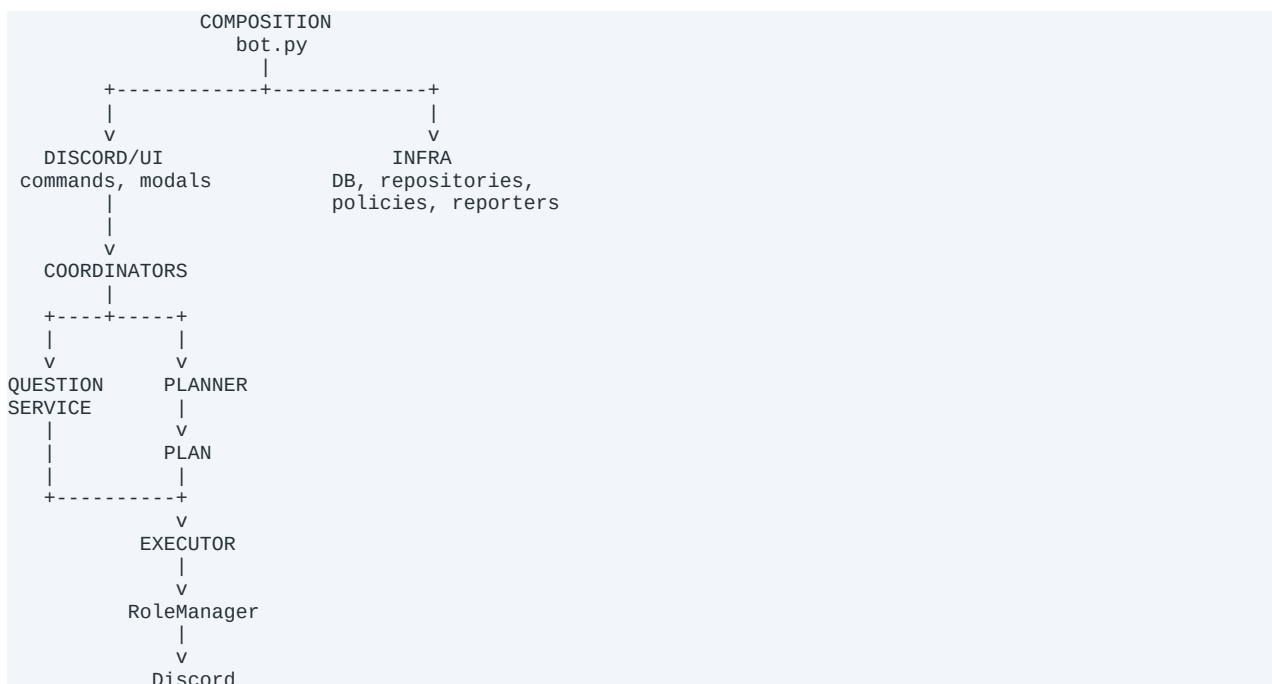
Snowflake

Identifiant numérique utilisé par Discord pour ses entités (guild, rôle, channel, utilisateur...).

39. Résumé : le modèle mental à conserver

Pour maintenir Claviger, il n'est pas nécessaire de mémoriser chaque fonction.

Il faut retenir les frontières.



Pour `/membre` et `/noctis`, le raisonnement essentiel reste :

1. Observer l'état réel
2. Construire le choix utilisateur
3. Recevoir l'état désiré

```

4. Revalider l'état courant
5. Calculer un plan
6. Prévalider les cibles
7. Appliquer les mutations
8. Retourner ce qui a réellement changé
9. Reporter les incidents significatifs

```

C'est cette séquence, plus que le nom exact des classes, qui permet de retrouver rapidement le bon fichier lorsqu'un bug apparaît.

L'architecture actuelle est donc lisible comme une série de traductions contrôlées :

```

Discord -> intention utilisateur
intention -> plan métier
plan -> mutations Discord
Discord -> snapshot
snapshot -> plan de synchronisation
plan de synchronisation -> transaction SQLite
événement -> reporters

```

Une intervention de maintenance reste saine tant qu'elle respecte ces frontières ou qu'elle les fait évoluer pour résoudre un besoin réel, plutôt que de les contourner pour gagner quelques lignes à court terme.

Annexe A - Carte rapide des fichiers à connaître

Fichier / zone	Question à laquelle il répond
main.py	Comment le processus démarre-t-il ?
bot.py	Quels composants existent et comment sont-ils câblés ?
config.py	D'où viennent token, guild ID, DB path et forum de report ?
commands/member_command.py	Quelles préconditions ouvrent /membre ?
ui/member_questionnaire_modal.py	Comment la sélection membre est-elle présentée/soumise ?
services/member_workflow_coordinator_service.py	Comment le cas d'usage /membre est-il orchestré ?
services/member_role_planner_service.py	Comment calcule-t-on les rôles à ajouter/retirer ?
services/member_role_executor_service.py	Comment le plan est-il validé puis appliqué ?
services/role_manager_service.py	Où se trouvent les appels effectifs add_roles/remove_roles ?
services/adult_access_workflow_service.py	Comment le catalogue Noctis devient-il des thèmes base + IA ?
services/noctis_role_planner_service.py	Comment la sélection Noctis devient-elle un plan ?
database/connection.py	Comment une connexion SQLite est-elle ouverte/fermée ?
database/schema.py	Comment le schéma est-il versionné et migré ?
database/status.py	Comment savoir si la DB est exploitable sans la modifier ?
repositories/role_channel_catalog_repository.py	Comment le catalogue est-il persisté ?
services/role_channel_discovery_service.py	Comment l'état Discord rôle/salon est-il photographié ?
services/catalog_sync_planner_service.py	Comment snapshot et DB sont-ils comparés ?
services/catalog_sync_coordinator_service.py	Où la transaction de sync globale est-elle pilotée ?
policies/	Quelles règles changent selon la guild ?
services/authorization.py	Qui peut exercer une capacité donnée ?
reporting/	Comment un événement devient-il log ou report Discord ?
tests/	Quelles garanties comportementales sont réellement protégées ?

Annexe B - Bibliothèques et primitives à reconnaître immédiatement

<code>discord.Client</code>	-> connexion/cycle de vie Discord
<code>app_commands.CommandTree</code>	-> registre des slash commands
<code>app_commands.Command</code>	-> slash command
<code>app_commands.Group</code>	-> groupe / sous-commandes
<code>discord.Interaction</code>	-> contexte d'une action utilisateur
<code>interaction.response</code>	-> première réponse à l'interaction
<code>interaction.followup</code>	-> réponses après defer/première réponse
<code>discord.ui.Modal</code>	-> formulaire modal
<code>discord.ui.CheckboxGroup</code>	-> groupe de cases à cocher
<code>discord.Embed</code>	-> message riche
<code>discord.Member.add_roles</code>	-> mutation Discord réelle
<code>aiosqlite.connect</code>	-> connexion SQLite asynchrone
<code>connection.execute</code>	-> requête SQL
<code>connection.commit/rollback</code>	-> transaction
<code>PRAGMA user_version</code>	-> version de schéma Claviger
<code>@dataclass</code>	-> model de données compact
<code>frozen=True</code>	-> immutabilité logique
<code>slots=True</code>	-> attributs stricts/compacts
<code>StrEnum</code>	-> valeurs nommées de type chaîne
<code>Protocol</code>	-> contrat structurel léger
<code>Generic / TypeVar</code>	-> repository générique typé
<code>@asynccontextmanager</code>	-> cycle de vie sûr d'une ressource async
<code>pytest.mark.asyncio</code>	-> test d'une coroutine
<code>AsyncMock</code>	-> mock awaitable

Annexe C - Etat documentaire

Ce guide décrit la branche V1 fournie pour l'audit. Il doit être relu à chaque refactoring structurant : changement du composition root, nouvelle frontière d'intégration, modification du modèle de catalogue, généralisation multi-guild, nouvelle stratégie d'autorisation ou introduction d'outils IA.

Les améliorations de logs et de gestion d'erreurs prévues après le premier déploiement modifieront surtout les boundaries et l'observabilité. Tant que le coeur **questionnaire -> planner -> executor** reste inchangé, la majorité du modèle mental présenté ici restera valable.